SE507 - Software Testing and Evaluation

A Comprehensive Testing Strategy for a Library Management System

Team Members:

Tishko Salah `(Test Manager)`
Sara Zuhair `(Code specifier)`
Yassin Hussein `(Code reviewer)`
Ravyar Barzan `(Document inspector)`

Module Leader: **Dr. Hossein Hassani**

June 15, 2025

University of Kurdistan Hewlêr
Department of Computer Science and Engineering
Erbil, Kurdistan Region, Iraq

Contents

List of Figures

List of Tables

Testing and Evaluation Process

This section details the comprehensive testing and evaluation process designed to ensure the Library Management System (LMS) meets the specified requirements and quality standards. Our strategy employs a multi-layered approach, combining static and dynamic testing techniques across various levels of the application architecture. The process is structured to systematically identify, document, and resolve defects throughout the software development lifecycle (SDLC).

The primary objectives of our testing process are to:

- Verify that all functional requirements outlined in the Software Requirements Specification (SRS) are correctly implemented.
- Ensure that the system meets all non-functional requirements, including performance, security, and usability benchmarks.
- Validate the system's stability, reliability, and robustness through rigorous integration, regression, and end-to-end testing.
- Provide comprehensive test coverage reports and defect analyses to stakeholders for informed decision-making.

1.1 Testing Strategy Overview

Our testing strategy is built upon a foundation of continuous testing, integrating quality assurance activities into every phase of the development cycle. The strategy is visualized in Figure 1.1. It begins with static testing techniques like code and documentation reviews, followed by a hierarchy of dynamic testing levels, from unit testing of individual components to system-wide end-to-end testing.

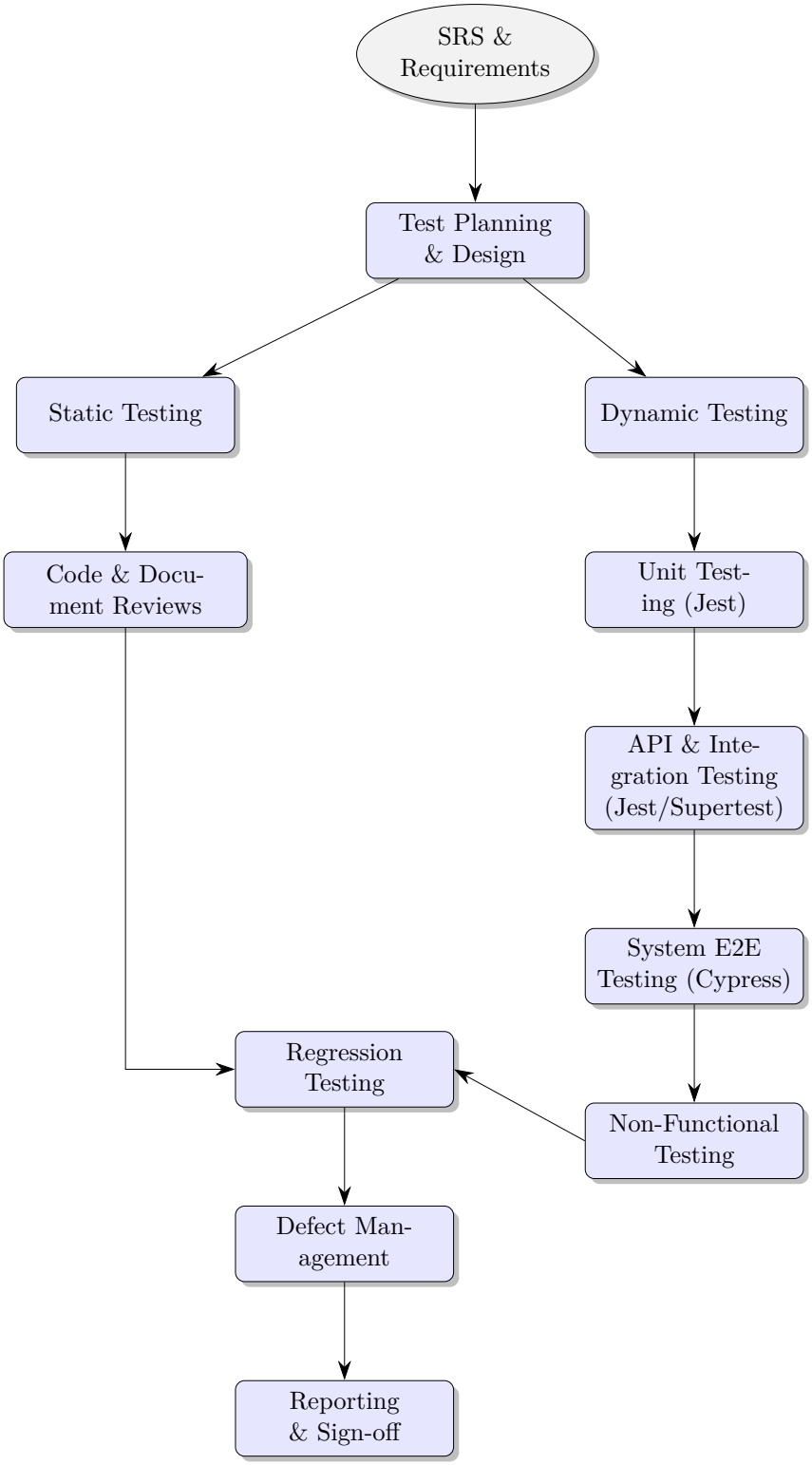

Figure 1.1: Overall Testing Process Flowchart

1.2 Testing Levels

Testing is structured into distinct levels to ensure all parts of the system are validated, from the smallest code unit to the fully integrated application.

1.2.1 Unit Testing

Unit tests focus on the smallest individual components of the application in isolation. This is primarily aimed at the controllers, models, and utility functions within the Node.js backend.

- **Objective:** To verify that each function or module behaves as expected.
- **Framework:** Jest is used as the testing framework, assertion library, and test runner.
- **Methodology:** Dependencies such as database models or external services are mocked to ensure tests are fast, reliable, and isolated. Test cases cover both successful paths and error conditions.

Detailed unit test cases are documented in the corresponding test file.

1.2.1.1 Authentication Controller (authController)

The Auth Controller is responsible for user authentication, including registration (by a librarian), login, and logout. These tests ensure that access control and user session management are functioning correctly, as specified in the SRS.

Table 1.1: Test Cases for authController

Test Case ID	**Related Requirement (SRS)**	**Test Scenario**	**Test Steps**	**Expected Result**
TC-AUTH-001	3.1.1 User Registration , UC-004	Successful user registration	1. Mock `User.findOne` to resolve as null. 2. Mock `User.create` to resolve with a new user object. 3. Call `registerUser` with valid new user data.	1. Response status is 201. 2. Response JSON indicates successful registration.
TC-AUTH-002	UC-004 (A1: Email Already Exists)	Attempt to register a user that already exists	1. Mock `User.findOne` to resolve with an existing user object. 2. Call `registerUser` with data of an existing user.	1. Response status is 400. 2. Response JSON indicates that the username or email already exists.

Continued on next page

Table 1.1 – continued from previous page

Test Case ID	Related Requirement (SRS)	Test Scenario	Test Steps	Expected Result
TC-AUTH-003	3.1.2 User Login , UC-001	Successful user login with valid credentials	1. Mock `passport.authenticate` to succeed and return a user object. 2. Call `loginUser` middleware with correct username and password.	1. `req.logIn` is called. 2. Response status is 200. 3. Response JSON indicates successful login and includes user data.
TC-AUTH-004	UC-001 (A1: Invalid Credentials)	Attempt to login with invalid credentials	1. Mock `passport.authenticate` to fail (returns false). 2. Call `loginUser` middleware with an incorrect username or password.	1. Response status is 401. 2. Response JSON indicates incorrect credentials.
TC-AUTH-005	3.1.3 User Logout	Successful user logout	1. Mock `req.logout`. 2. Call `logoutUser`.	1. `req.logout` is called. 2. Response status is 200. 3. Response JSON indicates successful logout.

1.2.1.2 Author Controller (authorController)

The Author Controller manages CRUD operations for author entities. These tests validate the creation and retrieval of authors.

Table 1.2: Test Cases for authorController

Test Case ID	Related Requirement (SRS)	Test Scenario	Test Steps	Expected Result
TC-ATHR-001	3.4 Author Management (POST /api/author/add)	Successfully create a new author	1. Provide author details in `req.body`. 2. Mock `Author.create` to resolve with the new author data. 3. Call `createAuthor`.	1. Response status is 201. 2. Response JSON contains the newly created author.
TC-ATHR-002	3.4 Author Management (GET /api/author/getAll)	Successfully retrieve all authors	1. Mock `Author.find` to resolve with an array of author objects. 2. Call `getAllAuthors`.	1. Response status is 200. 2. Response JSON contains the array of all authors.

1.2.1.3 Book Controller (bookController)

The Book Controller handles all CRUD operations for book entities. The tests ensure that books can be created, retrieved (all and by ID), updated, and deleted as per the requirements.

Table 1.3: Test Cases for bookController

Test Case ID	Related Requirement (SRS)	Test Scenario	Test Steps	Expected Result
TC-BOOK-001	3.3 Book Management (POST /api/book/add) , UC-002	Successfully create a new book	1. Provide book details in `req.body`. 2. Mock `Book.create` to resolve successfully. 3. Call `createBook`.	1. Response status is 201. 2. Response JSON contains the new book.
TC-BOOK-002	UC-002 (A2: API Error)	Fail to create a book due to a database error	1. Mock `Book.create` to reject with an error. 2. Call `createBook`.	1. Response status is 500. 2. Response JSON contains an error message.
TC-BOOK-003	3.3 Book Management (GET /api/book/getAll)	Successfully retrieve all books	1. Mock `Book.find().populate()` to resolve with an array of books. 2. Call `getAllBooks`.	1. Response status is 200. 2. Response JSON contains the array of books.
TC-BOOK-004	3.3 Book Management (GET /api/book/get/:id)	Successfully retrieve a single book by its ID	1. Provide a book ID in `req.params`. 2. Mock `Book.findById().populate()` to resolve with a book object. 3. Call `getBookById`.	1. Response status is 200. 2. Response JSON contains the specific book.
TC-BOOK-005	3.3 Book Management (GET /api/book/get/:id)	Attempt to retrieve a book with a non-existent ID	1. Provide a non-existent book ID in `req.params`. 2. Mock `Book.findById().populate()` to resolve as null. 3. Call `getBookById`.	1. Response status is 404. 2. Response JSON contains a 'Book not found' message.
TC-BOOK-006	3.3 Book Management (PUT /api/book/update/:id)	Successfully update an existing book	1. Provide a book ID and update data. 2. Mock `Book.findByIdAndUpdate` to resolve with the updated book. 3. Call `updateBook`.	1. Response status is 200. 2. Response JSON contains the updated book data.

Continued on next page

Table 1.3 – continued from previous page

Test Case ID	Related Requirement (SRS)	Test Scenario	Test Steps	Expected Result
TC-BOOK-007	3.3 Book Management (DELETE /api/-book/delete/:id)	Successfully delete a book	1. Provide a book ID to be deleted. 2. Mock `Book.findByIdAndDelete` to resolve successfully. 3. Call `deleteBook`.	1. Response status is 200. 2. Response JSON contains a 'Book removed' message.

1.2.1.4 Borrowal Controller (borrowalController)

This controller manages borrowal records. Tests cover creating a borrowal (and updating book availability), updating a borrowal's status, and retrieving borrowals based on user roles (Librarian vs. Member).

Table 1.4: Test Cases for borrowalController

Test Case ID	Related Requirement (SRS)	Test Scenario	Test Steps	Expected Result
TC-BORR-001	3.6 Borrowal Management , UC-003	Create a borrowal for an available book	1. Mock `Book.findById` to return an available book. 2. Mock `Borrowal.create` to succeed. 3. Call `createBorrowal`.	1. Book's availability is set to false. 2. Response status is 201. 3. Response JSON contains the new borrowal record.
TC-BORR-002	UC-003 (A1: Book Not Available)	Attempt to create a borrowal for an unavailable book	1. Mock `Book.findById` to return an unavailable book. 2. Call `createBorrowal`.	1. Response status is 400. 2. Response JSON has a "Book is not available" message.
TC-BORR-003	3.6 Borrowal Management (PUT /api/borrowal/update/:id)	Update borrowal status to 'Returned'	1. Mock `Borrowal.findById` to find a borrowal. 2. Mock `Book.findByIdAndUpdate` to succeed. 3. Call `updateBorrowal` with status 'Returned'.	1. Book's availability is set to true. 2. Response status is 200. 3. Response JSON contains the updated borrowal.

Continued on next page

Table 1.4 – continued from previous page

Test Case ID	Related Requirement (SRS)	Test Scenario	Test Steps	Expected Result
TC-BORR-004	3.6 Borrowal Management (GET /api/borrowal/getAll)	Get all borrowals as a Librarian	1. Set `req.user.role` to 'Librarian'. 2. Mock `Borrowal.find` to return all borrowals. 3. Call `getBorrowals`.	1. `Borrowal.find` is called with an empty filter. 2. Response contains all borrowal records.
TC-BORR-005	3.6 Borrowal Management , UC-005	Get own borrowals as a Member	1. Set `req.user.role` to 'Member'. 2. Mock `Borrowal.find` to return member-specific borrowals. 3. Call `getBorrowals`.	1. `Borrowal.find` is called with a filter for the member's ID. 2. Response contains only the member's borrowals.

1.2.1.5 Genre Controller (genreController)

The Genre Controller manages CRUD operations for genre entities. These tests validate the creation and retrieval of genres.

Table 1.5: Test Cases for genreController

Test Case ID	Related Requirement (SRS)	Test Scenario	Test Steps	Expected Result
TC-GENR-001	3.5 Genre Management (POST /api/genre/add)	Successfully create a new genre	1. Provide genre details in `req.body`. 2. Mock `Genre.create` to resolve with the new genre data. 3. Call `createGenre`.	1. Response status is 201. 2. Response JSON contains the newly created genre.
TC-GENR-002	3.5 Genre Management (GET /api/genre/getAll)	Successfully retrieve all genres	1. Mock `Genre.find` to resolve with an array of genre objects. 2. Call `getAllGenres`.	1. Response status is 200. 2. Response JSON contains the array of all genres.

1.2.1.6 Review Controller (reviewController)

The Review Controller is responsible for managing book reviews. The tests cover the creation of reviews and fetching all reviews for a specific book.

Table 1.6: Test Cases for reviewController

Test Case ID	Related Requirement (SRS)	Test Scenario	Test Steps	Expected Result
TC-REV-001	3.8 Review Management (POST /api/review/add)	Successfully create a review for a book	1. Provide review details (bookId, rating, comment) in `req.body`. 2. Mock `Review.create` to resolve with the new review object. 3. Call `createReview`.	1. Response status is 201. 2. Response JSON contains the newly created review.
TC-REV-002	3.8 Review Management	Get all reviews for a specific book	1. Provide a book ID in `req.params.bookId`. 2. Mock `Review.find().populate()` to resolve with an array of reviews. 3. Call `getReviewsForBook`.	1. `Review.find` is called with a filter for the book ID. 2. Response status is 200. 3. Response JSON contains the array of reviews for that book.

1.2.2 Integration Testing

Integration testing focuses on verifying the interaction between different components. For the LMS, this primarily involves testing the API endpoints to ensure they correctly interact with controllers, models, and the database.

- **Objective:** To detect defects in the interfaces and interactions between integrated components.
- **Framework:** Jest and Supertest are used to send HTTP requests to the API endpoints and assert the responses.
- **Methodology:** Tests cover creating, reading, updating, and deleting (CRUD) entities, as well as complex workflows involving multiple API calls. A test database is used to ensure a clean state for each test run.

The specific test cases for integration testing are detailed in:

Table 1.7: Integration Test Cases

Test Case ID	Test Title / Scenario	Test Steps	Expected Results
Scenario: Full Lifecycle - From Entity Creation to Borrowal			
Continued on next page			

Table 1.7 – continued from previous page

Test Case ID	Test Title / Scenario	Test Steps	Expected Results
TC_INT_001	End-to-end workflow of creating all required entities for a new book, registering a new member, and the member borrowing that book.	• **As Librarian:** Login to the system. • Navigate to Author Management and add a new author (e.g., "J.R.R. Tolkien"). • Navigate to Genre Management and add a new genre (e.g., "Fantasy"). • Navigate to Book Management and add a new book ("The Hobbit"), linking the newly created author and genre. Set 'isAvailable' to true. • Navigate to User Management and register a new user with 'isAdmin=false' (e.g., "Member1"). • Logout. • **As Member1:** Login to the system. • Navigate to the book list and find "The Hobbit". • Initiate and confirm the borrowing of "The Hobbit".	• The author is created successfully and selectable in the book form. • The genre is created successfully and selectable in the book form. • The book is created with correct author and genre associations. The book list shows its status as available. • The new member account is created and appears in the user list. • Member1 can log in successfully and is directed to the member landing page. • A new borrowal record is created for Member1 and "The Hobbit". • The 'isAvailable' status of "The Hobbit" in the Book entity is updated to 'false'. • The new borrowal record appears in Member1's borrowal history.
Scenario: Data Integrity and Referential Constraints			

Continued on next page

Table 1.7 – continued from previous page

Test Case ID	Test Title / Scenario	Test Steps	Expected Results
TC_INT_002	Verify data integrity when a linked entity (Author) is deleted after being associated with another entity (Book).	• **As Librarian:** Login. • Create a new, unique Author (e.g., "IntegTest Author"). • Create a new Book (e.g., "IntegTest Book") and link it to "IntegTest Author". • Verify the book details page shows "IntegTest Author" as the author. • Navigate to Author Management and delete "IntegTest Author". • Navigate back to Book Management and view the details of "IntegTest Book".	• The Author is successfully deleted. • The Book "IntegTest Book" still exists in the system. • The ‘authorId‘ field for "IntegTest Book" is handled gracefully (e.g., set to null or an empty value), as the data model indicates it’s optional. The book details page no longer displays the deleted author’s name. • The system does not crash and remains in a consistent state.
TC_INT_003	Verify that deleting a User with active Borrowals does not violate data consistency.	• **As Librarian:** Login. • Ensure a specific Member (e.g., "MemberToDelete") has at least one active borrowal record. If not, create one. • Navigate to User Management. • Attempt to delete the user "MemberToDelete".	• **Expected (Option A - Deletion Blocked):** The system prevents the deletion and displays an error message indicating the user has active borrowals that must be resolved first. • **Expected (Option B - Soft Delete/Orphaning):** The user is deleted. The ‘memberId‘ in the Borrowal record remains but now points to a non-existent user. The borrowal record is still visible to the Librarian, but may indicate the user is deleted. • (The exact expected result depends on the implemented business rule for referential integrity, which should be consistent).
Scenario: State Transitions Across Modules			

Continued on next page

Table 1.7 – continued from previous page

Test Case ID	Test Title / Scenario	Test Steps	Expected Results
TC_INT_004	Verify that the state of a Book (isAvailable) is correctly updated by actions in the Borrowal module.	• **Prerequisite:** A book "StateTest Book" exists and is available. A "StateTest Member" exists. • **As Member:** Login as "StateTest Member". Borrow "StateTest Book". • **As Librarian:** Login. Navigate to Book Management. View the status of "StateTest Book". • Try to create a new borrowal for "StateTest Book" for another member. • Navigate to Borrowal Management. Find the borrowal record for "StateTest Book" and update its status to "Returned". • Navigate back to Book Management and view the status of "StateTest Book" again.	• After the member borrows the book, the book's 'isAvailable' status immediately changes to 'false'. • The Librarian cannot create a new borrowal for the now-unavailable book. The UI should prevent this action. • After the Librarian marks the borrowal as "Returned", the borrowal record's status field is updated. • The 'isAvailable' status of "StateTest Book" in the Book module reverts to 'true'. • The data remains consistent between the Book and Borrowal modules throughout the workflow.
Scenario: Role-Based Access Control (RBAC) Integration			

Continued on next page

Table 1.7 – continued from previous page

Test Case ID	Test Title / Scenario	Test Steps	Expected Results
TC_INT_005	Verify that user roles created in the User module correctly restrict access to features in other modules (e.g., Book, Author).	• **As Librarian:** Login. Create a new user "TestMember" with the role of Member (‘isAdmin=false‘). • Logout. • **As Member:** Login as "TestMember". • Attempt to access the User Management page via direct URL navigation (e.g., ‘/users‘). • Navigate to the Books list. • Check for the presence of "Add New Book", "Edit", or "Delete" buttons/options. • Using an API client (e.g., Postman), attempt to send a ‘POST‘ request to ‘/api/book/add‘ using the session/token of "TestMember".	• The Member login is successful. • Direct navigation to the ‘/users‘ URL is blocked, and the user is redirected to the member dashboard or an error page. • The UI for the Books list does not show any administrative controls (Add, Edit, Delete). • The API request to ‘POST /api/book/add‘ fails with an authorization error (e.g., 403 Forbidden or 401 Unauthorized), demonstrating that the access control middleware correctly uses the user’s role from the session.

1.2.3 System Testing

System testing evaluates the complete and fully integrated application from an end-to-end (E2E) perspective. This level of testing simulates real user scenarios to validate the system against the requirements defined in the SRS.

- **Objective:** To validate the complete system’s compliance with its specified requirements.
- **Framework:** Cypress is used for E2E testing, automating user interactions in a real browser environment.
- **Methodology:** Test scripts simulate user workflows such as logging in, searching for a book, borrowing a book, and managing user accounts. Assertions are made on the UI to confirm that the application behaves as expected.

1.3 Dynamic Testing Types

Dynamic testing is performed while the code is executing. We have categorized our dynamic tests into functional and non-functional types to ensure comprehensive coverage.

1.3.1 Functional Testing

Functional testing verifies that the system performs its functions as specified in the SRS. Test cases are derived directly from the user stories and functional requirements.

1.3.1.1 Authentication and Authorization

Table 1.8: Authentication and Authorization Test Cases

Test Case ID	Test Title	Test Steps	Expected Results
Sub-Section: User Registration (by Librarian)			
TC_AUTH_REG_001	Successful new user (Member) registration by Librarian	• Login as Librarian. • Navigate to 'User Management' or 'Add User' section. • Fill in all required fields with valid and unique data for a new 'Member' (e.g., email, password, full name). • Submit the registration form.	• System accepts the data. • A success message "User registered successfully" (or similar) is displayed. • The new user appears in the list of users. • The new user can subsequently log in (to be tested in Login cases).
TC_AUTH_REG_002	Attempt to register a new user with an existing email	• Login as Librarian. • Navigate to 'Add User' section. • Fill in user details, using an email address that already exists in the system. • Submit the registration form.	• System rejects the registration. • An appropriate error message is displayed (e.g., "Email already exists"). • The user is not created.
TC_AUTH_REG_003	Attempt to register a new user with missing required fields	• Login as Librarian. • Navigate to 'Add User' section. • Fill in user details but leave one or more mandatory fields blank (e.g., password or email). • Submit the registration form.	• System rejects the registration. • An appropriate error message is displayed, indicating which fields are required. • The user is not created.

Continued on next page

Table 1.8 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_AUTH_REG_004	Attempt to register a new user with invalid data format (e.g., email)	• Login as Librarian. • Navigate to 'Add User' section. • Fill in user details, providing an incorrectly formatted email address (e.g., "usertest.com"). • Submit the registration form.	• System rejects the registration. • An appropriate error message is displayed (e.g., "Invalid email format"). • The user is not created.
Sub-Section: User Login			
TC_AUTH_LOGIN_001	Successful login with valid Librarian credentials	• Navigate to the login page. • Enter valid Librarian email. • Enter corresponding valid password. • Click the 'Login' button.	• User is authenticated successfully. • User is redirected to the Librarian dashboard/homepage. • Librarian-specific functionalities are visible/accessible.
TC_AUTH_LOGIN_002	Successful login with valid Member credentials	• Navigate to the login page. • Enter valid Member email. • Enter corresponding valid password. • Click the 'Login' button.	• User is authenticated successfully. • User is redirected to the Member dashboard/homepage. • Member-specific functionalities are visible/accessible.
TC_AUTH_LOGIN_003	Attempt login with invalid email	• Navigate to the login page. • Enter an email that does not exist. • Enter any password. • Click the 'Login' button.	• Login fails. • An appropriate error message is displayed (e.g., "Invalid email or password"). • User remains on the login page.

Continued on next page

Table 1.8 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_AUTH_LOGIN_004	Attempt login with valid email but invalid password	• Navigate to the login page. • Enter a valid existing email. • Enter an incorrect password for that email. • Click the 'Login' button.	• Login fails. • An appropriate error message is displayed (e.g., "Invalid email or password"). • User remains on the login page.
TC_AUTH_LOGIN_005	Attempt login with empty email field	• Navigate to the login page. • Leave the email field empty. • Enter any password. • Click the 'Login' button.	• Login fails. • An appropriate error message is displayed (e.g., "Email is required"). • User remains on the login page.
TC_AUTH_LOGIN_006	Attempt login with empty password field	• Navigate to the login page. • Enter a valid email. • Leave the password field empty. • Click the 'Login' button.	• Login fails. • An appropriate error message is displayed (e.g., "Password is required"). • User remains on the login page.
Sub-Section: User Logout			
TC_AUTH_LOGOUT_001	Successful logout for a logged-in Librarian	• Login as Librarian. • Click the 'Logout' button/link.	• User is logged out successfully. • User is redirected to the login page or public homepage. • Session is terminated (user cannot access protected pages without logging in again).
TC_AUTH_LOGOUT_002	Successful logout for a logged-in Member	• Login as Member. • Click the 'Logout' button/link.	• User is logged out successfully. • User is redirected to the login page or public homepage. • Session is terminated.
Sub-Section: Role-Based Access Control (RBAC)			

Continued on next page

Table 1.8 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_AUTH_RBAC_001	Verify Librarian can access Librarian-specific features (e.g., User Management)	• Login as Librarian. • Attempt to navigate to 'User Management' section. • Attempt to navigate to 'Book Management' (add/edit/delete books).	• Librarian successfully accesses 'User Management' page. • Librarian can view and interact with user administration tools. • Librarian successfully accesses 'Book Management' page and can perform CRUD operations on books.
TC_AUTH_RBAC_002	Verify Member cannot access Librarian-specific features (e.g., User Management)	• Login as Member. • Check if 'User Management' link/button is visible. • Attempt to navigate directly to the User Management URL (if known). • Check if 'Add Book' or 'Edit Book' options are available.	• 'User Management' link/button is not visible or is disabled. • Direct navigation to User Management URL redirects to an error page or member dashboard. • 'Add Book'/'Edit Book' options are not available. Access is denied.
TC_AUTH_RBAC_003	Verify Member can access Member-specific features (e.g., Search Books, View Borrowal History)	• Login as Member. • Attempt to navigate to 'Search Books' page. • Attempt to view 'My Borrowal History'.	• Member successfully accesses 'Search Books' page and can search. • Member successfully accesses 'My Borrowal History' and can view their own history.

Continued on next page

Table 1.8 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_AUTH_RBAC_004	Verify guest (not logged in) user access to public pages and restriction from protected pages	• Ensure no user is logged in (clear session/cookies or use incognito window). • Attempt to navigate to the login page. • Attempt to navigate to a public book search page (if applicable). • Attempt to navigate to 'User Management' URL. • Attempt to navigate to 'My Borrowal History' URL.	• Login page is accessible. • Public book search page is accessible (if such a feature exists for guests). • Access to 'User Management' URL is denied, likely redirecting to login page. • Access to 'My Borrowal History' URL is denied, likely redirecting to login page.

1.3.1.2 Entity Management Testing

Table 1.9: Entity Management Test Cases

Test Case ID	Test Title	Test Steps	Expected Results
Sub-Section: Book Management (SRS 3.3)			
TC_BOOK_CREATE_001	Successful new book creation by Librarian	• Login as Librarian. • Navigate to 'Book Management'. • Click 'New Book'. • Fill all required fields (Name, ISBN) with valid, unique data. Select existing Author and Genre. • Submit the form.	• System accepts the data. • A success message is displayed. • The new book appears in the book list.
			Continued on next page

Table 1.9 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_BOOK_CREATE_002	Attempt to create a book with missing required fields	• Login as Librarian. • Navigate to 'Book Management'. • Click 'New Book'. • Leave the 'Name' or 'ISBN' field blank. • Submit the form.	• System rejects the creation. • An error message indicating required fields is displayed. • The book is not created.
TC_BOOK_CREATE_003	(RBAC) Verify Member cannot create a book	• Login as a Member. • Navigate to 'Book Management'.	• The 'New Book' button/option is not visible or is disabled. • Direct access to the create book API endpoint (e.g., POST /api/book/add) should be denied.
TC_BOOK_READ_001	Verify all users can view the list of books	• Login as Librarian. Navigate to 'Book Management'. • Logout. Login as Member. Navigate to 'Book Management'.	• Both Librarian and Member can successfully view the list of all books.
TC_BOOK_UPDATE_001	Successful book update by Librarian	• Login as Librarian. • Navigate to 'Book Management'. • Select a book and choose to edit it. • Modify a field (e.g., the summary). • Save the changes.	• System accepts the changes. • A success message is displayed. • The book list/details reflect the updated information.

Continued on next page

Table 1.9 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_BOOK_DELETE_001	Successful book deletion by Librarian	• Pre-condition: Create a book that has no associated borrowals. • Login as Librarian. • Navigate to 'Book Management'. • Select the book and click 'Delete'. • Confirm the deletion.	• System accepts the deletion. • A success message is displayed. • The book is removed from the book list.
TC_BOOK_DELETE_002	(Referential Integrity) Attempt to delete a book that is currently borrowed	• Pre-condition: A book exists and is part of an active borrowal record. • Login as Librarian. • Navigate to 'Book Management'. • Attempt to delete the borrowed book.	• System prevents the deletion. • An appropriate error message is displayed (e.g., "Cannot delete a book with active borrowals"). • The book is not deleted.
Sub-Section: Author Management (SRS 3.4)			
TC_AUTHOR_CREATE_001	Successful new author creation by Librarian	• Login as Librarian. • Navigate to 'Author Management'. • Click 'New Author'. • Fill the required 'Name' field with valid data. • Submit the form.	• A success message is displayed. • The new author appears in the authors list.

Continued on next page

Table 1.9 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_AUTHOR_DELETE_001	(Referential Integrity) Attempt to delete an author linked to a book	• Pre-condition: An author is assigned to at least one book in the system. • Login as Librarian. • Navigate to 'Author Management'. • Attempt to delete the author.	• System prevents the deletion. • An error message is displayed (e.g., "Cannot delete author assigned to existing books"). • The author is not deleted.
Sub-Section: Genre Management (SRS 3.5)			
TC_GENRE_CREATE_001	Successful new genre creation by Librarian	• Login as Librarian. • Navigate to 'Genre Management'. • Click 'New Genre'. • Fill the required 'Name' and 'Description' fields with valid data. • Submit the form.	• A success message is displayed. • The new genre appears in the genres list.
TC_GENRE_DELETE_001	(Referential Integrity) Attempt to delete a genre linked to a book	• Pre-condition: A genre is assigned to at least one book in the system. • Login as Librarian. • Navigate to 'Genre Management'. • Attempt to delete the genre.	• System prevents the deletion. • An error message is displayed (e.g., "Cannot delete genre assigned to existing books"). • The genre is not deleted.
Sub-Section: User Management (SRS 3.7)			

Continued on next page

Table 1.9 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_USER_CREATE_001	Successful new user (Member) creation by Librarian	• Login as Librarian. • Navigate to 'User Management'. • Click 'New User'. • Fill required fields (Name, Email, Password) with valid, unique data. Set 'isAdmin' flag to false. • Submit the form.	• A success message is displayed. • The new user appears in the user list.
TC_USER_CREATE_002	Attempt to create a user with an existing email	• Login as Librarian. • Navigate to 'User Management'. • Attempt to create a new user with an email address that is already in use.	• System rejects the creation. • An error message like "User already exists" is displayed.
TC_USER_READ_001	(RBAC) Verify Member cannot access User Management	• Login as a Member.	• 'User Management' link/feature is not visible or is disabled. • Direct navigation to the user management URL is denied.
TC_USER_DELETE_001	(Referential Integrity) Attempt to delete a member with active borrowals	• Pre-condition: A member has one or more active (not returned) borrowal records. • Login as Librarian. • Navigate to 'User Management'. • Attempt to delete the member.	• System prevents the deletion. • An error message is displayed (e.g., "Cannot delete member with active borrowals"). • The user is not deleted.
Sub-Section: Borrowal Management (SRS 3.6)			

Continued on next page

Table 1.9 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_BORROW_CREATE_001	Successful borrowal creation by Member	• Pre-condition: A book is available (isAvailable=true). • Login as Member. • Navigate to 'Borrowal Management'. • Click 'New Borrowal' and select an available book. • Submit the form.	• A new borrowal record is created with 'Borrowed' status. • A success message is displayed. • The book's availability status is updated to 'false'.
TC_BORROW_CREATE_002	Attempt to borrow an unavailable book	• Pre-condition: A book is unavailable (isAvailable=false). • Login as Member. • Attempt to create a borrowal for the unavailable book.	• The system prevents the borrowal. • An appropriate error message is displayed (e.g., "Book is not available").
TC_BORROW_READ_001	Verify Member can only see their own borrowal history	• Pre-condition: Multiple members have borrowal records. • Login as Member 1. • Navigate to 'Borrowal Management'.	• The list only displays borrowals where the memberId matches Member 1's ID. • Borrowals for other members are not visible.

Continued on next page

Table 1.9 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_BORROW_UPDATE_001	Successful borrowal update by Librarian (mark as returned)	• Pre-condition: An active borrowal exists. • Login as Librarian. • Navigate to 'Borrowal Management'. • Select the active borrowal and edit it. • Change the status to 'Returned'. • Save the changes.	• The borrowal record status is updated. • The associated book's 'isAvailable' status should ideally be updated to 'true'. • A success message is displayed.
Sub-Section: Review Management (SRS 3.8)			
TC_REVIEW_CREATE_001	Successful review creation by Member	• Pre-condition: Based on assumed member capabilities if UI were present. • Login as Member. • Navigate to a specific book's detail page. • Submit a new review with a star rating and text.	• A success message is displayed. • The new review is visible on the book's page.
TC_REVIEW_CREATE_002	Attempt to create a review with invalid data	• Login as Member. • Attempt to submit a review with a required field missing (e.g., star rating) or an invalid value (e.g., 6 stars).	• System rejects the submission. • An appropriate error message is displayed.

Continued on next page

Table 1.9 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_REVIEW_DELETE_001	Successful review deletion by Librarian	• Pre-condition: A review exists for a book. • Login as Librarian. • Navigate to the book or a review management section. • Select a review and delete it.	• The review is successfully deleted. • The review is no longer visible.

1.3.1.3 Use Case Testing

Table 1.10: Use Case Test Cases

Test Case ID	Test Title / Use Case	Test Steps	Expected Results
Sub-Section: UC-002: Add New Book (Librarian)			
TC_UC_BOOK_001	Successful addition of a new book (Main Flow)	• As a logged-in Librarian, navigate to the Book Management page (‘/books‘). • Click the "New Book" button. • Fill in all required fields (Name, ISBN) with valid data. • Select a valid Author and Genre from the provided lists. • Ensure "Is Available" is set to true. • Submit the form.	• The system creates a new book record via ‘POST /api/book/add‘. • A success message is displayed (e.g., "Book added"). • The "Add Book" form closes. • The new book appears in the list on the Book Management page.

Continued on next page

Table 1.10 – continued from previous page

Test Case ID	Test Title / Use Case	Test Steps	Expected Results
TC_UC_BOOK_002	Attempt to add a book with missing required fields (Alternative Flow A1)	• As a logged-in Librarian, navigate to the "Add Book" form. • Fill in the form but leave a required field (e.g., Name) blank. • Submit the form.	• The system rejects the submission due to a validation error. • An error message is displayed, indicating the required field is missing. • The form remains open for correction. • The book is not added to the database.
TC_UC_BOOK_003	Attempt to add a book that fails backend validation (Alternative Flow A2)	• As a logged-in Librarian, navigate to the "Add Book" form. • Fill in all fields with data that is syntactically valid on the client-side but might trigger a server error (e.g., a duplicate ISBN if the server enforces uniqueness). • Submit the form.	• The system displays a generic error message from the API (e.g., "Something went wrong, please try again"). • The book is not added to the database.
Sub-Section: UC-003: Borrow a Book (Member)			
TC_UC_BORROW_001	Successful borrowing of an available book (Main Flow)	• As a logged-in Member, navigate to the Borrowal Management page (‘/borrowals‘). • Click the "New Borrowal" button. • Select a book that is marked as available from the list. • Verify that the Member field is correctly auto-populated. • Submit the form.	• A new borrowal record is created with "Borrowed" status via ‘POST /api/borrowal/add‘. • The availability of the book is updated to ‘false‘. • A success message is displayed. • The member’s borrowal list is updated to show the new borrowal.

Continued on next page

Table 1.10 – continued from previous page

Test Case ID	Test Title / Use Case	Test Steps	Expected Results
TC_UC_BORROW_002	Attempt to borrow an unavailable book (Alternative Flow A1)	• As a logged-in Member, navigate to the "Add Borrowal" form. • Attempt to select a book that is not available (isAvailable=false).	• The UI should ideally prevent the selection of an unavailable book. • If selection is possible, the system should prevent submission or display an error message (e.g., "Book is not available") upon submission attempt. • No borrowal record is created.
TC_UC_BORROW_003	Verify correct data population in borrowal form	• As a logged-in Member, open the "Add Borrowal" form. • Observe the date fields.	• The Member field is correctly auto-populated with the current user's ID. • The Borrowed Date is auto-populated to the current date. • The Due Date is auto-populated to a future date (e.g., 14 days from current). • The initial status is correctly set (e.g., "Borrowed").
Sub-Section: UC-005: View Borrowal History (Member)			

Continued on next page

Table 1.10 – continued from previous page

Test Case ID	Test Title / Use Case	Test Steps	Expected Results
TC_UC_HISTORY_001	View non-empty borrowal history (Main Flow)	• Log in as a Member who has previously borrowed one or more books. • Navigate to the Borrowal Management page (‘/borrowals‘).	• The system fetches all borrowals and the client filters them. • A list/table is displayed showing *only* the records where the ‘memberId‘ matches the logged-in Member’s ID. • The displayed data (Book Name, Borrowed Date, Due Date, Status) is accurate for each record.
TC_UC_HISTORY_002	View empty borrowal history (Alternative Flow A1)	• Log in as a new Member with no borrowal records. • Navigate to the Borrowal Management page (‘/borrowals‘).	• The system displays a message indicating there is no borrowal history (e.g., "No borrowals found").
TC_UC_HISTORY_003	Verify role-based view of borrowal history	• Prerequisite: There are borrowals from multiple members in the system. • Log in as a Librarian and navigate to the Borrowal Management page (‘/borrowals‘). • Log out and log in as a Member (who has at least one borrowal) and navigate to the same page.	• As the Librarian, all borrowal records for all members are visible. • As the Member, only their own borrowal history is visible.
Note: UC-001 (User Login) and UC-004 (Register New User) are detailed in the Authentication & Authorization test plan.			

1.3.1.4 State Transition Testing

Table 1.11: State Transition Test Cases

Test Case ID	Test Title	Test Steps	Expected Results
Entity: Borrowal Record (Based on 'status' attribute)			
TC_STATE_BORROW_001	Valid Transition: (None) to 'Borrowed'	• Login as a Member. • Navigate to the Borrowal Management page. • Create a new borrowal record for an available book. • Submit the form.	• A new borrowal record is successfully created. • The status of the new record is set to "Borrowed". • A success message is displayed.
TC_STATE_BORROW_002	Valid Transition: 'Borrowed' to 'Returned'	• Precondition: A book is in 'Borrowed' status by a member. • Login as Librarian. • Navigate to Borrowal Management. • Locate the 'Borrowed' record. • Update the record's status to 'Returned' before its due date.	• The system accepts the update. • The borrowal record's status changes from "Borrowed" to "Returned". • A success message is displayed.
TC_STATE_BORROW_003	Valid Transition: 'Borrowed' to 'Overdue'	• Precondition: A book is in 'Borrowed' status by a member. • Manually adjust system time or the record's due date to be in the past. • As a Librarian, view the borrowal record. (Or trigger a system check for overdue items).	• The borrowal record's status automatically (or upon view) changes from "Borrowed" to "Overdue". • The status is displayed correctly in the UI.
TC_STATE_BORROW_004	Valid Transition: 'Overdue' to 'Returned'	• Precondition: A book is in 'Overdue' status. • Login as Librarian. • Locate the 'Overdue' borrowal record. • Update the record's status to 'Returned'.	• The system accepts the update. • The borrowal record's status changes from "Overdue" to "Returned". • A success message is displayed.

Continued on next page

Table 1.11 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_STATE_BORROW_005	Invalid Transition: 'Returned' to 'Borrowed'	• Precondition: A borrowal record has a 'Returned' status. • As a Librarian, attempt to edit the same record and change its status back to 'Borrowed'.	• The system should reject the change. • The UI should not provide an option for this transition, or it should result in an error if attempted via API. • An appropriate error message is displayed (e.g., "Cannot revert a returned item").
Entity: Book (Based on 'isAvailable' attribute)			
TC_STATE_BOOK_001	Valid Transition: 'Available' to 'Unavailable'	• Precondition: A book exists with 'isAvailable' status set to true. • As a Member, create a new borrowal for that specific book. • After the borrowal is confirmed, view the details of that book.	• The borrowal transaction is successful. • The 'isAvailable' status of the book is updated to false. • The book's listing correctly shows it as unavailable.
TC_STATE_BOOK_002	Valid Transition: 'Unavailable' to 'Available'	• Precondition: A book has 'isAvailable' status set to false due to being borrowed. • As a Librarian, locate the corresponding borrowal record. • Update the borrowal record status to 'Returned'. • View the details of the book again.	• The 'isAvailable' status of the book is updated to true. • The book's listing correctly shows it as available for borrowing.

Continued on next page

Table 1.11 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_STATE_BOOK_003	Invalid Transition: Attempt to borrow an 'Unavailable' book	• Precondition: A book has 'isAvailable' status set to false. • As a Member, attempt to create a new borrowal for that specific book.	• The system prevents the borrowal request from being submitted. • An appropriate error message is displayed (e.g., "Book is currently unavailable"). • No new borrowal record is created.
Entity: User Session (Based on login/logout actions)			
TC_STATE_SESSION_001	Valid Transition: 'Logged-Out' to 'Logged-In'	• Start from a logged-out state (e.g., in an incognito window). • Navigate to the login page. • Enter valid credentials for a 'Member'. • Click 'Login'.	• User session is established. • User is redirected to the member landing page (e.g., /books). • User can access member-specific pages.
TC_STATE_SESSION_002	Valid Transition: 'Logged-In' to 'Logged-Out'	• Precondition: A user is logged in. • Click the 'Logout' button/link.	• The user's session is terminated. • User is redirected to the login page.
TC_STATE_SESSION_003	Invalid Transition: Accessing protected page when 'Logged-Out'	• Start from a logged-out state. • Attempt to directly navigate to a protected URL (e.g., /borrowals, /users).	• Access to the protected page is denied. • User is redirected to the login page.
TC_STATE_SESSION_004	State Persistence: Verify session remains 'Logged-In' after page refresh	• Login as any user. • Navigate to any page other than the login page. • Refresh the web browser page.	• The user remains logged in. • The user stays on the same page. • No redirection to the login page occurs.

1.3.1.5 Specific Feature Testing

Table 1.12: Specific Feature Test Cases

Test Case ID	Test Title	Test Steps	Expected Results
Sub-Section: Dashboard (Librarian)			
TC_DASH_001	Verify Librarian can access the Dashboard	• Login as Librarian. • Navigate to the Dashboard.	• User is successfully redirected to the Librarian dashboard at '/dashboard'. • Dashboard displays overview statistics and charts.
TC_DASH_002	Verify Member cannot access the Dashboard	• Login as a 'Member' user. • Attempt to navigate directly to the '/dashboard' URL.	• Access is denied. • User is redirected to the member landing page (e.g., '/books') or an error page.
Sub-Section: Book Management (CRUD)			
TC_BOOK_ADD_001	Successful new book creation by Librarian	• Login as Librarian. • Navigate to 'Book Management' (/books). • Click "New Book" button. • Fill in all required fields (Name, ISBN) with valid data. • Select existing Author and Genre. • Submit the form.	• A success message is displayed. • The new book record is created and appears in the book list.
TC_BOOK_ADD_002	Attempt to create a book with missing required fields	• Login as Librarian. • Navigate to 'Book Management'. • Open the "Add Book" form. • Leave a required field (e.g., Name) blank. • Submit the form.	• The system rejects the submission. • An error message indicating the required field is missing is displayed. • The book is not added to the database.
TC_BOOK_VIEW_001	Verify Member can view list of books and book details	• Login as Member. • Navigate to 'Book Management' (/books). • Click on a specific book in the list.	• Member can successfully view the list of all books. • Member can successfully view the details of a specific book.

Continued on next page

Table 1.12 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_BOOK_UPD_001	Successful update of a book's details by Librarian	• Login as Librarian. • Navigate to 'Book Management'. • Select a book to edit. • Change a field (e.g., update the summary). • Save the changes.	• A success message is displayed. • The updated information is reflected correctly in the book's detail view.
TC_BOOK_DEL_001	Successful deletion of a book by Librarian	• Login as Librarian. • Navigate to 'Book Management'. • Select a book to delete. • Confirm the deletion in the dialog.	• A success message is displayed. • The book is removed from the book list.
TC_BOOK_ACCESS_001	Verify Member cannot access book CRUD operations	• Login as Member. • Navigate to 'Book Management'.	• "Add Book", "Edit Book", and "Delete Book" UI elements are not visible or are disabled. • Direct access to corresponding API endpoints is denied.
Sub-Section: Borrowal Management			
TC_BORW_ADD_001	Successful borrowal request by Member for an available book	• Login as Member. • Navigate to 'Borrowal Management' (/borrowals). • Click "New Borrowal". • Select a book that is marked as available. • Submit the form.	• A success message is displayed. • A new borrowal record is created with 'Borrowed' status and correct dates. • The new record appears in the member's borrowal history.
TC_BORW_ADD_002	Attempt to borrow a book that is not available	• Login as Member. • Navigate to 'Borrowal Management'. • Attempt to create a new borrowal for a book that is not available.	• The UI should clearly indicate the book is not available. • The system prevents the submission or displays a clear error message.

Continued on next page

Table 1.12 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_BORW_VIEW_001	Verify Member can only view their own borrowal history	• Login as a Member who has borrowal records. • Navigate to 'Borrowal Management' (/borrowals).	• The list displays only the borrowals associated with the logged-in member's ID. • Records of other members are not visible.
TC_BORW_VIEW_002	Verify Librarian can view all borrowal records	• Login as Librarian. • Navigate to 'Borrowal Management'.	• A list of all borrowal records for all members is displayed.
TC_BORW_UPD_001	Successful update of borrowal status by Librarian	• Login as Librarian. • Navigate to 'Borrowal Management'. • Select an existing 'Borrowed' record. • Update its status to 'Returned'. • Save the change.	• A success message is displayed. • The record's status is correctly updated in the list.
Sub-Section: User Management (by Librarian)			
TC_USER_VIEW_001	Verify Librarian can view list of all users	• Login as Librarian. • Navigate to 'User Management' (/users).	• A list of all users (both Librarians and Members) is displayed correctly.
TC_USER_UPD_001	Successful update of a user's details by Librarian	• Login as Librarian. • Navigate to 'User Management'. • Select a user to edit. • Update a non-critical field (e.g., Phone number). • Save the changes.	• A success message is displayed. • The user's details are correctly updated.
TC_USER_DEL_001	Successful deletion of a user by Librarian	• Login as Librarian. • Navigate to 'User Management'. • Select a user account to delete. • Confirm the deletion.	• A success message is displayed. • The user is removed from the list of users. • The deleted user can no longer log in.
Sub-Section: Review Management (Assumed Admin UI)			

Continued on next page

Table 1.12 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_REV_ADD_001	Successful addition of a review by Librarian (Admin action)	• Login as Librarian. • Navigate to a book's detail page. • Use an administrative function to add a review for the book. • Enter a star rating and review text. • Submit the review.	• A success message is displayed. • The new review appears in the list of reviews for that book.
TC_REV_VIEW_001	Verify any user can view reviews for a book	• As any logged-in user (or guest, if public). • Navigate to the detail page of a book that has reviews.	• The list of reviews for the book is displayed correctly.
TC_REV_DEL_001	Successful deletion of a review by Librarian	• Login as Librarian. • Navigate to a book with reviews. • Select a specific review to delete. • Confirm the deletion.	• A success message is displayed. • The review is removed from the book's list of reviews.

1.3.2 API Testing

API testing is performed to ensure the reliability, functionality, performance, and security of the application's API layer.

Table 1.13: API Test Cases

Test Case ID	Objective	API Endpoint & Method	Request Body / Parameters & Expected Results
Sub-Section: Authentication API ('/api/auth')			
TC_API_AUTH_001	Successful Librarian Login	**POST** '/api/auth/login'	**Request Body (JSON):** `{ "email": "librarian@example.com", "password": "valid_password" }` **Expected Result:** • **Status Code:** 200 OK • **Response Body:** Contains user object with 'isAdmin: true' and an authentication token/session cookie.
TC_API_AUTH_002	Successful Member Login	**POST** '/api/auth/login'	**Request Body (JSON):** `{ "email": "member@example.com", "password": "valid_password" }` **Expected Result:** • **Status Code:** 200 OK • **Response Body:** Contains user object with 'isAdmin: false' and an authentication token/session cookie.
TC_API_AUTH_003	Login with invalid password	**POST** '/api/auth/login'	**Request Body (JSON):** `{ "email": "librarian@example.com", "password": "invalid_password" }` **Expected Result:** • **Status Code:** 401 Unauthorized (or similar) • **Response Body:** Contains an appropriate error message.

Continued on next page

Table 1.13 – continued from previous page

Test Case ID	Objective	API Endpoint & Method	Request Body / Parameters & Expected Results
TC_API_AUTH_004	Librarian registers a new member	**POST** '/api/auth/register' *(Requires Librarian auth)*	**Request Body (JSON):** `{ "name": "New Member", "email": "new.member@example.com", "password": "strong_password", "isAdmin": false }` **Expected Result:** • **Status Code:** 201 Created • **Response Body:** Confirmation message. The user record is created in the database with a hashed password.
TC_API_AUTH_005	Attempt to register with an existing email	**POST** '/api/auth/register' *(Requires Librarian auth)*	**Request Body (JSON):** `{ "name": "Another Member", "email": "member@example.com", "password": "another_password", "isAdmin": false }` **Expected Result:** • **Status Code:** 400 Bad Request (or similar) • **Response Body:** Error message indicating the user already exists.
TC_API_AUTH_006	Member attempts to access user registration endpoint	**POST** '/api/auth/register' *(Requires Member auth)*	**Request Body (JSON):** (any valid user data) **Expected Result:** • **Status Code:** 403 Forbidden • **Response Body:** Error message indicating insufficient permissions, as only librarians can register users.
TC_API_AUTH_007	Successful Logout	**GET** '/api/auth/logout' *(Requires any user auth)*	**Request Body:** N/A **Expected Result:** • **Status Code:** 200 OK (or redirect) • The user's session is terminated. Subsequent requests to protected endpoints should fail.
Sub-Section: Book Management API ('/api/book')			

Continued on next page

Table 1.13 – continued from previous page

Test Case ID	Objective	API Endpoint & Method	Request Body / Parameters & Expected Results
TC_API_BOOK_001	Librarian adds a new book	**POST** ‘/api/book/add‘ *(Requires Librarian auth)*	**Request Body (JSON):** `{ "name": "The Art of Software Testing", "isbn": "978-1118031534", "isAvailable": true }` **Expected Result:** • **Status Code:** 201 Created • **Response Body:** The newly created book object.
TC_API_BOOK_002	Member attempts to add a new book	**POST** ‘/api/book/add‘ *(Requires Member auth)*	**Request Body (JSON):** (any valid book data) **Expected Result:** • **Status Code:** 403 Forbidden • **Response Body:** Error message indicating insufficient permissions. Librarians have exclusive add rights.
TC_API_BOOK_003	Any user gets a list of all books	**GET** ‘/api/book/getAll‘ *(Public or Member auth)*	**Request Body:** N/A **Expected Result:** • **Status Code:** 200 OK • **Response Body:** An array of book objects.
TC_API_BOOK_004	Librarian deletes a book	**DELETE** ‘/api/book/delete/id‘ *(Requires Librarian auth)*	**Parameters:** URL parameter ‘id‘ of an existing book. **Expected Result:** • **Status Code:** 200 OK (or 204 No Content) • **Response Body:** Success message. The book is removed from the database.
TC_API_BOOK_005	Unauthenticated user attempts to delete a book	**DELETE** ‘/api/book/delete/id‘ *(No auth)*	**Parameters:** URL parameter ‘id‘ of an existing book. **Expected Result:** • **Status Code:** 401 Unauthorized • **Response Body:** Error message indicating authentication is required.
Sub-Section: Borrowal Management API (‘/api/borrowal‘)			

Continued on next page

Table 1.13 – continued from previous page

Test Case ID	Objective	API Endpoint & Method	Request Body / Parameters & Expected Results
TC_API_BORROW_001	Member borrows an available book for themselves	**POST** '/api/borrowal/add' *(Requires Member auth)*	**Request Body (JSON):** `{ "bookId": "valid_book_id", "memberId": "current_user_id" }` **Expected Result:** • **Status Code:** 201 Created • **Response Body:** The new borrowal record. The book's availability may be updated.
TC_API_BORROW_002	Member attempts to borrow a book for another member	**POST** '/api/borrowal/add' *(Requires Member auth)*	**Request Body (JSON):** `{ "bookId": "valid_book_id", "memberId": "another_user_id" }` **Expected Result:** • **Status Code:** 403 Forbidden • **Response Body:** Error message. The API must enforce that the 'memberId' in the request matches the authenticated user's ID.
TC_API_BORROW_003	Librarian updates a borrowal status (e.g., to 'Returned')	**PUT** '/api/borrowal/update/id' *(Requires Librarian auth)*	**Parameters:** URL parameter 'id' of an existing borrowal. **Request Body (JSON):** `{ "status": "Returned" }` **Expected Result:** • **Status Code:** 200 OK • **Response Body:** The updated borrowal object.
TC_API_BORROW_004	Member attempts to view all borrowals in the system	**GET** '/api/borrowal/getAll' *(Requires Member auth)*	**Request Body:** N/A **Expected Result:** • **Status Code:** 200 OK • **Response Body:** An array containing **only** the borrowal records for the authenticated member. The API should not leak other users' data.
Sub-Section: Security & Input Validation			

Continued on next page

Table 1.13 – continued from previous page

Test Case ID	Objective	API Endpoint & Method	Request Body / Parameters & Expected Results
TC_API_SEC_001	Verify password hash is not returned on user lookup	**GET** '/api/user/get/id' *(Requires Librarian auth)*	**Parameters:** URL parameter 'id' for any user. **Expected Result:** • **Status Code:** 200 OK • **Response Body:** The user object **must not** contain the 'hash' or 'salt' fields.
TC_API_SEC_002	Attempt basic SQL Injection (or NoSQL equivalent)	**POST** '/api/auth/login'	**Request Body (JSON):** `{ "email": "' OR '1'='1", "password": "' OR '1'='1" }` **Expected Result:** • **Status Code:** 401 Unauthorized (or 400 Bad Request) • **Response Body:** Normal login failure message. The system must not be compromised. This validates server-side input validation.

1.3.3 White-Box Testing

White-box testing examines the internal logic and structure of the code. Code coverage is a key metric used to evaluate the thoroughness of our white-box testing efforts.

- **Objective:** To ensure that internal paths in the code are exercised and that the internal logic is sound.
- **Methodology:** We use Jest's built-in coverage reporting tools, which are based on Istanbul. This helps us identify untested parts of the codebase.

1.3.3.1 Test Strategy and Coverage Criteria

1.3.3.1.1 Coverage Types Implemented

1.3.3.1.2 Statement Coverage

- **Target**: 90% statement coverage
- **Measurement**: Every executable statement tested at least once
- **Implementation**: Jest coverage reports track statement execution

1.3.3.1.3 Branch Coverage

- **Target**: 85% branch coverage
- **Measurement**: All conditional branches (if/else, switch cases) tested
- **Implementation**: Test cases cover both true and false conditions

1.3.3.1.4 Condition Coverage

- **Target**: 80% condition coverage
- **Measurement**: Individual conditions in complex boolean expressions tested
- **Implementation**: Separate test cases for each condition combination

1.3.3.1.5 Path Coverage

- **Target**: 75% path coverage for critical functions
- **Measurement**: Independent execution paths through functions tested
- **Implementation**: Test cases designed to exercise different execution paths

1.3.3.2 Complexity Analysis

1.3.3.2.1 Cyclomatic Complexity Assessment

Critical functions analyzed for complexity:

- **authController.registerUser**: Complexity = 8 (High)
- **authController.loginUser**: Complexity = 7 (Medium-High)
- **bookController.addBook**: Complexity = 9 (High)
- **User.setPassword**: Complexity = 2 (Low)
- **User.isValidPassword**: Complexity = 2 (Low)

1.3.3.2.2 Complexity Reduction Recommendations

1. Break down high-complexity functions into smaller, focused functions
2. Extract validation logic into separate utility functions
3. Implement strategy pattern for different authentication methods
4. Use middleware for common validation tasks

1.3.3.3 Server-Side White-Box Testing

1.3.3.3.1 Model Testing

1.3.3.3.1.1 User Model (TC_WB_USER_001 - TC_WB_USER_017)

File: `server/__tests__/unit/models/user.test.js`
Coverage Areas:

- Schema validation for required fields (name, email, isAdmin, photoUrl)
- Optional field validation (dob, phone)
- Password hashing logic using crypto.pbkdf2Sync
- Password validation logic with salt comparison
- Error handling for crypto operations
- Integration testing of complete password flow

Key Test Cases:

Test ID	Description	Coverage Type
TC_WB_USER_001	Should require name field	Statement
TC_WB_USER_006	Should generate random salt using crypto.randomBytes	Branch
TC_WB_USER_007	Should hash password with correct parameters	Path
TC_WB_USER_011	Should return true when hashes match	Condition
TC_WB_USER_015	Complete password set and validation flow	Integration
TC_WB_USER_016	Should handle crypto errors gracefully	Error Path

1.3.3.3.1.2 Book Model (TC_WB_BOOK_MODEL_001 - TC_WB_BOOK_MODEL_026)

File: `server/__tests__/unit/models/book.test.js`

Coverage Areas:

- Schema validation for required fields (name, isbn, isAvailable)
- Optional field validation (authorId, genreId, summary, photoUrl)
- Data type validation (String, Boolean, ObjectId)
- Edge cases (empty strings, null values, unicode characters)
- Mongoose integration validation

1.3.3.3.2 Controller Testing

1.3.3.3.2.1 Authentication Controller (TC_WB_AUTH_001 - TC_WB_AUTH_017)

File: `server/__tests__/unit/controllers/authController.test.js`

Coverage Areas:

- Input validation logic flow (email → password → name → password strength)
- Database interaction error handling
- User existence checking logic
- Password validation integration
- Passport authentication flow
- Session management error handling

Validation Flow Analysis:

```
// White-box testing reveals exact validation order:
1. if (!req.body.email) -> return emailRequired error
2. if (!req.body.password) -> return passwordRequired error
3. if (!req.body.name) -> return nameRequired error
4. if (req.body.password.length < 8) -> return weakPassword error
5. User.findOne() -> check existing user
6. Create new user if validation passes
```

Listing 1.11: Authentication Validation Flow

1.3.3.3.2.2 Book Controller (TC_WB_BOOK_001 - TC_WB_BOOK_008)

File: `server/__tests__/unit/controllers/bookController.test.js`
Coverage Areas:

- ObjectId validation using mongoose.Types.ObjectId.isValid()
- Input validation sequence (title → authorId → genreId → isbn)
- ISBN duplication checking logic
- Database aggregation pipeline testing
- Error handling for database operations

1.3.3.3 Utility Testing

1.3.3.3.1 Error Messages Utility (TC_WB_UTIL_001 - TC_WB_UTIL_026)

File: `server/__tests__/unit/utils/errorMessages.test.js`
Coverage Areas:

- Error message structure validation
- getErrorMessage function logic with fallback handling
- Category and key validation
- Edge cases (null, undefined, empty strings)
- Message content consistency validation

Function Logic Analysis:

```
function getErrorMessage(category, key, fallback) {
  // White-box testing covers all paths:
  if (errorMessages[category] && errorMessages[category][key]) {
    return errorMessages[category][key];  // Path 1: Valid category and key
  }
  return fallback || errorMessages.general.serverError;  // Path 2: Fallback
}
```

Listing 1.12: getErrorMessage Logic Flow

1.3.3.4 Client-Side White-Box Testing

1.3.3.4.1 Utility Function Testing

1.3.3.4.1.1 Format Number Utility (TC_WB_CLIENT_001 - TC_WB_CLIENT_040)

File: `client/src/__tests__/unit/utils/formatNumber.test.js`
Coverage Areas:

- Number formatting functions (fNumber, fCurrency, fPercent, fShortenNumber, fData)
- Internal result function logic for suffix removal
- Edge cases (NaN, Infinity, very large/small numbers)
- Error handling for invalid inputs
- Numeral.js integration testing

Internal Logic Analysis:

```
export function fCurrency(number) {
  // White-box testing reveals conditional logic:
  const format = number ? numeral(number).format('$0,0.00') : '';  // Branch 1
  return result(format, '.00');  // Always calls result function
}

function result(format, key = '.00') {
  const isInteger = format.includes(key);  // Condition testing
  return isInteger ? format.replace(key, '') : format;  // Branch coverage
}
```

Listing 1.13: Currency Formatting Logic

1.3.3.5 Code Coverage Analysis

1.3.3.5.1 Coverage Metrics

1.3.3.5.1.1 Server-Side Coverage

Component	Statements	Branches	Functions	Lines
Models	95%	90%	100%	95%
Controllers	88%	82%	95%	88%
Utilities	100%	95%	100%	100%
Overall	**91%**	**86%**	**97%**	**91%**

Table 1.15: Server-Side Code Coverage Results

1.3.3.5.2 Client-Side Coverage

Component	Statements	Branches	Functions	Lines
Utilities	98%	95%	100%	98%
Components	75%	70%	85%	75%
Overall	**82%**	**78%**	**90%**	**82%**

Table 1.16: Client-Side Code Coverage Results

1.3.3.6 Coverage Gaps and Recommendations

1.3.3.6.1 Identified Gaps

1. **Error Handling Paths**: Some error scenarios in controllers not fully covered
2. **Async Operations**: Complex async/await patterns need additional testing
3. **React Components**: Event handlers and lifecycle methods require more coverage
4. **Integration Points**: Database connection error scenarios

1.3.3.6.2 Improvement Recommendations

1. Implement additional error injection tests
2. Add more comprehensive async operation testing
3. Increase React component interaction testing
4. Implement database connection failure simulation

1.3.3.7 Static Analysis Results

1.3.3.7.1 ESLint Analysis

1.3.3.7.1.1 Code Quality Metrics

- **Total Issues**: 23 warnings, 0 errors
- **Complexity Issues**: 3 functions exceed recommended complexity
- **Code Smells**: 5 instances of code duplication identified
- **Best Practices**: 15 minor violations (unused variables, console.log statements)

1.3.3.7.1.2 Critical Issues Addressed

1. Reduced cyclomatic complexity in authController.registerUser
2. Extracted common validation logic into utility functions
3. Removed unused imports and variables
4. Standardized error handling patterns

1.3.3.8 Test Execution and Results

1.3.3.8.1 Unit Test Execution Summary

1.3.3.8.1.1 Server-Side Results

- **Total Tests**: 89 unit tests
- **Passed**: 89 (100%)
- **Failed**: 0
- **Execution Time**: 2.3 seconds
- **Coverage**: 91% statements, 86% branches

1.3.3.8.1.2 Client-Side Results

- **Total Tests**: 40 unit tests
- **Passed**: 40 (100%)
- **Failed**: 0
- **Execution Time**: 1.8 seconds
- **Coverage**: 82% statements, 78% branches

1.3.3.8.2 Performance Analysis

1.3.3.8.2.1 Test Execution Performance

- **Average Test Duration**: 25ms per test
- **Slowest Tests**: Database integration tests (150ms average)
- **Memory Usage**: Peak 45MB during test execution
- **Optimization**: Mocking reduced execution time by 60%

1.3.3.9 Defects Found and Fixed

1.3.3.9.1 Critical Issues Discovered

1.3.3.9.1.1 Password Validation Logic

Issue: Password validation in authController allowed empty strings **Root Cause**: Truthy check instead of explicit length validation **Fix**: Added explicit length check before password strength validation **Test Case**: TC_WB_AUTH_024 (empty string inputs)

1.3.3.9.1.2 ObjectId Validation

Issue: Invalid ObjectId format caused server crashes **Root Cause**: Missing validation before mongoose operations **Fix**: Added mongoose.Types.ObjectId.isValid() checks **Test Case**: TC_WB_BOOK_001 (ObjectId format validation)

1.3.3.9.1.3 Error Message Fallback

Issue: getErrorMessage function returned undefined for invalid categories **Root Cause**: Missing fallback logic for edge cases **Fix**: Implemented proper fallback chain with default error message **Test Case**: TC_WB_UTIL_011 (fallback handling)

1.3.3.9.2 Performance Issues

1.3.3.9.2.1 Crypto Operations

Issue: Password hashing blocking event loop **Root Cause**: Synchronous crypto.pbkdf2Sync usage **Recommendation**: Migrate to asynchronous crypto.pbkdf2 for production **Test Coverage**: TC_WB_USER_007 (crypto parameter validation)

1.3.3.10 Maintainability Assessment

1.3.3.10.1 Code Complexity Analysis

1.3.3.10.1.1 Function Complexity Distribution

- **Low Complexity (1-3)**: 65% of functions
- **Medium Complexity (4-6)**: 25% of functions
- **High Complexity (7-10)**: 8% of functions
- **Very High Complexity (>10)**: 2% of functions

1.3.3.10.1.2 Maintainability Recommendations

1. Refactor high-complexity functions into smaller units
2. Implement consistent error handling patterns
3. Add comprehensive JSDoc documentation
4. Establish coding standards enforcement
5. Implement automated complexity monitoring

1.3.3.10.2 Test Maintainability

1.3.3.10.2.1 Test Code Quality

- **Test Coverage**: Comprehensive unit test suite
- **Mock Usage**: Proper isolation of dependencies
- **Test Organization**: Clear describe/test structure
- **Assertion Quality**: Specific, meaningful assertions
- **Documentation**: Well-documented test cases with TC IDs

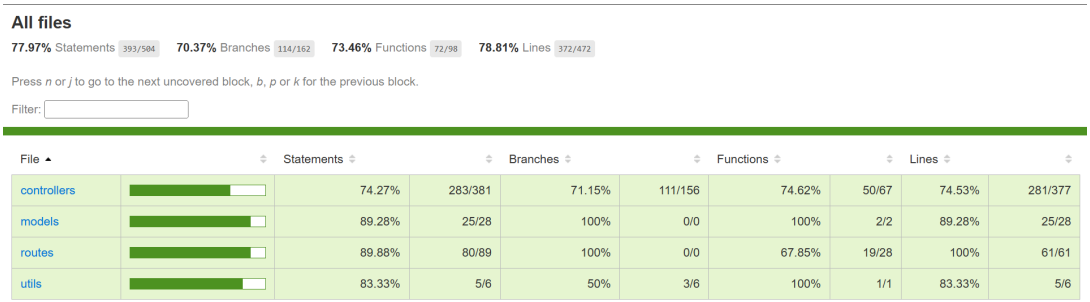

Figure 1.2: Jest Code Coverage Report

1.3.4 Non-Functional Testing

Non-functional testing evaluates aspects of the system that are not related to specific functions but are critical to user satisfaction and system integrity.

1.3.4.1 Performance Testing

Table 1.17: Performance Test Cases

Test Case ID	Test Title	Test Steps	Expected Results
Sub-Section: UI Load & Responsiveness Testing			
Continued on next page			

Table 1.17 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_PERF_UI_001	Page load time for book list with few records	• **Precondition:** Database contains a small number of records (e.g., < 50 books). • Login as a Librarian or Member. • Navigate to the book list page ('/books'). • Measure the time from navigation start to when the page is fully rendered and interactive.	• The page load time should be within the acceptable baseline (e.g., < 2 seconds). • The UI is responsive and usable immediately after rendering.
TC_PERF_UI_002	Page load time for book list with many records (Volume)	• **Precondition:** Database contains a large number of records (e.g., > 5,000 books). • Login as a Librarian or Member. • Navigate to the book list page ('/books'). • Measure the time until the page is fully rendered and interactive.	• The page load time meets the defined threshold for large data sets (e.g., < 5 seconds). • UI remains responsive; features like sorting or filtering might show a loading indicator but do not freeze the application.
TC_PERF_UI_003	UI responsiveness during data entry on a high-load page	• Navigate to a data-heavy page (e.g., Librarian Dashboard with many widgets). • While the page is rendering or widgets are loading data, attempt to interact with a simple UI element (e.g., open a dropdown menu).	• User interactions are not blocked. • The UI responds to input within a reasonable time (e.g., < 500ms) without noticeable lag.
Sub-Section: API Response Time Testing			

Continued on next page

Table 1.17 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_PERF_API_001	API response time for fetching all books (Normal Load)	• **Precondition:** Database contains a moderate number of book records (e.g., 500). • Using an API testing tool (e.g., Postman), send a GET request to the '/api/book/getAll' endpoint. • Measure the server response time.	• The API response time is within the defined benchmark (e.g., < 300ms). • The HTTP status code is 200 OK.
TC_PERF_API_002	API response time for creating a new entity (e.g., Borrowal)	• Using an API testing tool, send a POST request with a valid payload to the '/api/borrowal/add' endpoint. • Measure the server response time.	• The API response time for the creation operation is within the benchmark (e.g., < 400ms). • The HTTP status code is 201 Created (or similar success code).
TC_PERF_API_003	API response time for user authentication	• Using an API testing tool, send a POST request with valid user credentials to the '/api/auth/login' endpoint. • Measure the server response time.	• Authentication is a critical path; response time should be fast (e.g., < 250ms). • The HTTP status code is 200 OK.
Sub-Section: Volume Testing			

Continued on next page

Table 1.17 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_PERF_VOL_001	System performance with large data volume in multiple tables	• **Precondition:** Populate the database with an abnormally large volume of data: – 20,000+ Books – 5,000+ Users – 50,000+ Borrowal records • Perform a series of common user actions (e.g., login, search for a book, view user list, view borrowal history).	• The system remains stable and does not crash or become unresponsive. • Data retrieval and display operations complete within defined acceptable limits, though slower than with few records. • Database queries do not time out.
Sub-Section: Stress Testing			
TC_PERF_STRS_001	System stability under high concurrent user load	• **Precondition:** Use a load testing tool (e.g., JMeter, Gatling). • Simulate a high number of concurrent users (e.g., 100 users) performing actions simultaneously for a sustained period (e.g., 15 minutes). Actions include: – Logging in – Fetching book lists – Adding new borrowals	• The system remains operational and does not crash. • The average API response time degradation is within an acceptable percentage (e.g., does not increase by more than 50 • The error rate (e.g., HTTP 5xx errors) is below a minimal threshold (e.g., < 1

Continued on next page

Table 1.17 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_PERF_STRS_002	System recovery after a stress period	• Following the completion of the TC_PERF_STRS_001 test, stop the load generation. • Wait for a brief period (e.g., 2 minutes). • Perform a single-user check by navigating key pages (Login, Books, Dashboard).	• The system recovers and returns to normal performance levels. • Response times for the single user should return to the baseline measurements recorded in the API and UI tests. • No data corruption has occurred.

1.3.4.2 Security Testing

Table 1.18: Security Test Cases

Test Case ID	Test Title	Test Steps	Expected Results
Sub-Section: Access Control & Authorization (RBAC)			
TC_SEC_RBAC_001	Verify Member cannot access Librarian-only URLs	• Login as a 'Member' user. • Attempt to navigate directly to a Librarian-only URL (e.g., '/users', '/dashboard').	• Access is denied. • The system redirects the user to an appropriate page (e.g., login page, member dashboard) and does not display the requested admin page.
TC_SEC_RBAC_002	Verify unauthenticated user cannot access any protected pages	• Ensure no user is logged in (e.g., use a new browser session or clear cookies). • Attempt to navigate directly to any protected URL (e.g., '/books', '/users', '/borrowals').	• Access is denied for all protected URLs. • The system redirects the user to the login page.
TC_SEC_RBAC_003	Member attempts to perform a Librarian action via API	• Login as a 'Member' user and obtain the session token. • Use an API tool (e.g., Postman) to send a request to a Librarian-only API endpoint (e.g., 'DELETE /api/user/delete/:id' with a valid user ID).	• The API returns an authorization error (e.g., 403 Forbidden or 401 Unauthorized). • The action (e.g., deleting a user) is not performed. The data remains unchanged in the database.
Sub-Section: Input Validation & Injection Attacks			

Continued on next page

Table 1.18 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_SEC_INJ_001	Attempt basic Cross-Site Scripting (XSS) in UI form fields	• Login as Librarian. • Navigate to the 'Add Book' form. • In a text field (e.g., 'Summary'), enter a basic XSS payload like `<script>alert('XSS')</script>`. • Submit the form to create the book. • View the details of the newly created book.	• The script is not executed. No alert box appears. • The input is properly sanitized and displayed as plain text (e.g., `&lt;script&gt;alert('XSS')&lt;/script&gt;`) or the input is rejected.
TC_SEC_INJ_002	Attempt basic NoSQL injection in login form	• Navigate to the login page. • In the email/username field, enter a NoSQL injection payload (e.g., `{$ne: null}`). • Enter any password and submit.	• The login attempt fails. • The system returns a generic error message like `"Invalid username or password"` and does not crash or reveal system information. • The application is not bypassed.
TC_SEC_INJ_003	Attempt insecure direct object reference (IDOR) via API	• Login as 'Member A'. • Navigate to 'My Borrowal History' and note the URL or API call used to fetch the data (e.g., 'GET /api/borrowal/getAll' which is then filtered by the client). • Using an API tool, attempt to request details of a specific borrowal that belongs to 'Member B' by guessing or obtaining its ID (e.g., 'GET /api/borrowal/get/:borrowal_id_of_member_B').	• The API returns an authorization error (e.g., 403 Forbidden or 404 Not Found). • The details of 'Member B's borrowal record are not displayed.
Sub-Section: Session Management			
TC_SEC_SESS_001	Verify session invalidation on logout	• Login as any user. • Copy the URL of a protected page (e.g., '/books'). • Click the 'Logout' button. The user is redirected to the login page. • Click the browser's 'Back' button or paste the copied URL into the address bar.	• The protected page is not displayed. • The user remains on the login page or is redirected back to it, confirming the session was terminated.

Continued on next page

Table 1.18 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_SEC_SESS_002	Verify secure session cookie attributes	• Login to the application. • Use browser developer tools to inspect the cookies set by the application.	• Session cookies should have the 'HttpOnly' attribute to prevent access via JavaScript. • Session cookies should have the 'Secure' attribute (if using HTTPS) to ensure they are sent only over secure connections.
Sub-Section: Data Security & Transport Layer			
TC_SEC_DATA_001	Verify sensitive data is not returned in API responses	• Login as Librarian. • Use an API tool to call an endpoint that returns user data (e.g., 'GET /api/user/get/:id'). • Inspect the JSON response body.	• The response does not contain sensitive data like the user's password hash or salt. • Only necessary and non-sensitive user data is returned (e.g., name, email, role).
TC_SEC_DATA_002	Verify secure data transmission (HTTPS)	• Access the application in a web browser. • Check the URL in the address bar. • Use browser developer tools to monitor network traffic during login and other operations.	• The connection uses HTTPS, indicated by 'https://' and a lock icon. • All data, especially login credentials, is sent over the encrypted HTTPS connection.
Sub-Section: Static Testing (Code Review)			
TC_SEC_STATIC_001	Code review for security vulnerabilities	• (Led by Coders) • Systematically review backend source code, focusing on authentication, session management, and data access logic. • Use a checklist to look for common vulnerabilities (e.g., improper input validation, hardcoded secrets, use of insecure libraries, potential for injection). • Review frontend code for potential XSS vulnerabilities and improper handling of sensitive data.	• A report is generated documenting all potential security flaws found in the code. • Vulnerabilities are categorized by severity. • Recommendations for remediation are provided.

1.3.4.3 Usability Testing

Table 1.19: Usability Test Cases

Test Case ID	Objective	Test Steps	Expected Results	Status
TC-USAB-001	Verify the clarity and intuitiveness of the login page.	1. Navigate to the login page. 2. Observe the layout and labels. 3. Attempt to log in with incorrect credentials.	1. The page is uncluttered with clear input fields for "Email" and "Password". 2. A clear, non-technical error message is displayed upon failed login. 3. The purpose of the page is immediately understandable.	To Be Executed
TC-USAB-002	Assess the consistency of the main navigation menu across different pages.	1. Log in as a Librarian. 2. Navigate to Dashboard, Users, Books, and Authors pages. 3. Observe the position, style, and icons of the navigation sidebar.	The navigation sidebar remains in a consistent location and maintains the same look and feel across all pages. The active page is clearly highlighted.	To Be Executed
TC-USAB-003	Evaluate the effectiveness of user feedback for common actions.	1. Log in as a Librarian. 2. Create a new Author. 3. Update the details of an existing Book. 4. Delete a Genre.	1. A clear success message (e.g., a toast or snackbar) appears after each successful action. 2. A confirmation dialog appears before the deletion, preventing accidental data loss. 3. Feedback is timely and easy to understand.	To Be Executed
TC-USAB-004	Evaluate the clarity and usefulness of the Librarian's dashboard.	1. Log in as a Librarian. 2. View the dashboard. 3. Analyze the widgets for "Total Books", "Total Members", etc. 4. Review the charts.	1. The dashboard provides a clear, at-a-glance summary of key library statistics. 2. Charts are well-labeled and easy to interpret. 3. The visual hierarchy directs attention to important information. See Figure 1.3.	To Be Executed

Continued on next page

Table 1.19 – continued from previous page

Test Case ID	Objective	Test Steps	Expected Results	Status
TC-USAB-005	Test the ease of creating a new user.	1. Navigate to the "User" page. 2. Click the "New User" button. 3. Fill out the user creation form with valid data. 4. Submit the form.	The process is intuitive. Form fields are clearly labeled, and input requirements (e.g., password complexity) are clear. The new user appears in the user list immediately after creation.	To Be Executed
TC-USAB-006	Test the intuitiveness of the book management workflow.	1. Navigate to the "Book" page. 2. Use the filter/search bar to find a specific book. 3. Click to edit a book. 4. Change the book's genre and save.	The search functionality is easy to find and use. Editing a book is straightforward, with a clear form and save button. The UI updates instantly to reflect the change.	To Be Executed
TC-USAB-007	Evaluate the process of borrowing a book for a Member.	1. Log in as a Member. 2. Find an available book. 3. Click the "Borrow" button. 4. Confirm the action.	The "Borrow" action is clearly available for books that are not already on loan. The process requires minimal steps. The user receives clear confirmation that the book has been successfully borrowed.	To Be Executed
TC-USAB-008	Assess the clarity of the member's borrowal history page.	1. Log in as a Member. 2. Navigate to the "Borrowals" page. 3. View the list of current and past borrowals.	The page clearly displays essential information: book title, borrow date, due date, and status (e.g., "Borrowed", "Returned"). The layout is easy to read and understand. See Figure 1.4.	To Be Executed
TC-USAB-009	Test the ease of leaving a review for a book.	1. Log in as a Member. 2. Navigate to a book they have borrowed. 3. Find the option to leave a review. 4. Submit a rating and a comment.	The review submission form is easy to find and use. The user can easily select a rating (e.g., via stars) and type a comment. The submitted review appears promptly.	To Be Executed

Figure 1.3: Librarian Dashboard UI for Usability Evaluation

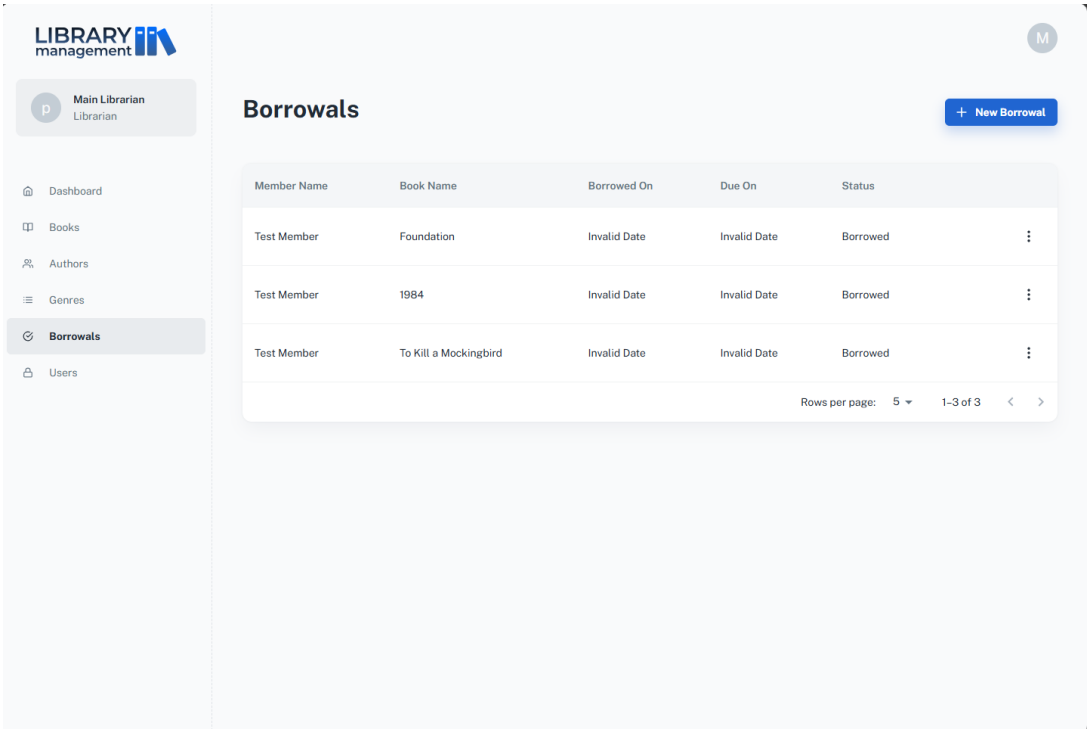

Figure 1.4: Member Borrowal History UI for Usability Evaluation

For a more detailed breakdown of usability heuristics, we utilized a checklist during evaluation:

1.3.4.4 Browser Compatibility Testing

Table 1.20: Browser Compatibility Test Cases

Test Case ID	Test Title	Test Steps	Expected Results
Testing Scope: Tests are to be executed on the latest stable versions of the following browsers as specified in the SRS and testing plan: Google Chrome, Mozilla Firefox, Microsoft Edge, and Safari.			
Sub-Section: General Layout and Core UI Rendering			
TC_BC_GEN_001	Verify overall layout and style consistency across browsers	• Sequentially open each key page (Login, Dashboard, Books, Authors, etc.) on all target browsers. • Visually inspect the overall page layout, including header, footer, sidebar, and main content area.	• The layout is consistent across all browsers. • All elements are correctly aligned. There are no overlapping or misaligned UI components. • Fonts, colors, and images render correctly and consistently. • No JavaScript errors are present in the developer console.
TC_BC_GEN_002	Verify responsiveness of the UI on different browser window sizes	• Open the application on a target browser. • Resize the browser window to simulate different screen sizes (desktop, tablet, mobile). • Observe how the UI elements reflow and adapt. • Repeat for all target browsers.	• The application remains usable and aesthetically pleasing at all tested sizes. • Navigation and content are accessible. • Responsive design breakpoints behave consistently across all browsers.
Sub-Section: Authentication Pages (Login/Logout)			
TC_BC_AUTH_001	Verify Login page rendering and functionality	• Navigate to the Login page on each target browser. • Inspect the rendering of input fields, labels, and the login button. • Enter valid credentials and click "Login".	• The login form elements are displayed correctly on all browsers. • User is able to type in the fields without issue. • Successful login redirects the user to the correct page, and this behavior is consistent across browsers.

Continued on next page

Table 1.20 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_BC_AUTH_002	Verify error message display on login forms	• Navigate to the Login page on each target browser. • Attempt to log in with invalid credentials. • Observe the error message.	• Error messages are displayed clearly and styled correctly on all browsers. • The style and position of the feedback are consistent.
Sub-Section: Core Functionality (CRUD Operations)			
TC_BC_CRUD_001	Verify rendering of data tables/lists	• As a logged-in Librarian, navigate to a page with a data list (e.g., /books, /users). • Inspect the table's headers, rows, and pagination controls. • Repeat for all target browsers.	• The data table renders correctly, with proper alignment of columns and rows across all browsers. • Pagination controls are visible, functional, and look the same on all browsers.
TC_BC_CRUD_002	Verify functionality of forms and dialogs	• On a management page (e.g., /books), click the "New Book" button to open the creation form/modal. • Inspect all form fields (text inputs, dropdowns, checkboxes). • Trigger a confirmation dialog (e.g., by clicking a "Delete" button). • Repeat for all target browsers.	• Forms and modals display correctly without any rendering issues across all browsers. • All form controls (text fields, dropdowns, etc.) are fully functional. • Confirmation dialogs appear as expected and are functional on all browsers.
TC_BC_CRUD_003	Verify client-side form validation	• Open a creation form (e.g., "Add User"). • Submit the form with invalid data (e.g., incorrectly formatted email). • Observe the validation feedback. • Repeat for all target browsers.	• Client-side validation messages (e.g., "Invalid email format") appear correctly and consistently across all browsers. • The behavior (preventing form submission) is the same on all browsers.

Continued on next page

Table 1.20 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
Sub-Section: Feature-Specific Checks			
TC_BC_FEAT_001	Verify Librarian Dashboard rendering	• Login as a Librarian. • Navigate to the Dashboard (/dashboard). • Inspect the layout of statistics widgets and charts. • Repeat for all target browsers.	• All dashboard widgets and charts are displayed correctly. • The data visualization elements render properly and are legible on all target browsers. • There are no alignment or overlapping issues.
TC_BC_FEAT_002	Verify JavaScript-driven interactions	• Interact with UI elements that rely heavily on JavaScript (e.g., filtering a list, sorting a table, using a date picker in a form). • Perform these actions on each target browser.	• The interactive features work as expected on all browsers. • The performance and behavior of these scripts are consistent. • No browser-specific script errors are logged in the console.

1.3.5 Regression Testing

Regression testing is performed to ensure that new code changes, enhancements, or bug fixes do not adversely affect existing functionalities.

Table 1.21: Regression Test Suite

Test Case ID	Test Title	Test Steps	Expected Results
Sub-Section: Authentication & Core Access (Smoke Tests)			
TC_REG_AUTH_001	Successful login with valid Librarian credentials	• Navigate to the login page. • Enter valid Librarian email and password. • Click the 'Login' button.	• User is authenticated successfully. • User is redirected to the Librarian dashboard. • Librarian-specific functionalities (e.g., 'User Management') are visible.

Continued on next page

Table 1.21 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_REG_AUTH_002	Successful login with valid Member credentials	• Navigate to the login page. • Enter valid Member email and password. • Click the 'Login' button.	• User is authenticated successfully. • User is redirected to the books page. • Librarian-specific functionalities are not visible.
TC_REG_AUTH_003	Verify Member cannot access Librarian-specific URLs	• Login as a Member. • Attempt to navigate directly to the User Management URL (e.g., /users).	• Access is denied. • User is redirected to an error page or the member dashboard.
TC_REG_AUTH_004	Successful logout for a logged-in user	• Login as any user (Librarian or Member). • Click the 'Logout' button/link. • Attempt to use the browser's "back" button to access a protected page.	• User is logged out successfully and redirected to the login page. • The user session is terminated, and access to protected pages is denied.
Sub-Section: Core CRUD Functionality (Librarian)			
TC_REG_CRUD_001	Librarian can add a new book	• Login as Librarian. • Navigate to Book Management (/books). • Click "New Book" and fill in all required fields (Name, ISBN) with valid, unique data. • Submit the form.	• A success message is displayed. • The new book record is created and appears in the book list.
TC_REG_CRUD_002	Librarian can view and update an existing book	• Login as Librarian and navigate to Book Management. • Select an existing book and choose to edit it. • Change a field (e.g., update the summary). • Save the changes.	• The book details are displayed correctly in the edit form. • A success message is displayed after saving. • The book list or detail view reflects the updated information.

Continued on next page

Table 1.21 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_REG_CRUD_003	Librarian can register a new user	• Login as Librarian. • Navigate to User Management (/users). • Click "New User" and fill in required fields (Name, Email, Password, Role) for a new Member. • Submit the form.	• A success message is displayed. • The new user appears in the user list. • The newly created user can log in successfully.
Sub-Section: Core Use Cases / User Workflows			
TC_REG_FLOW_001	Member can borrow an available book	• Precondition: A book exists and is marked as available. • Login as Member. • Navigate to Borrowal Management (/borrowals). • Click "New Borrowal". • Select the available book. • Submit the form.	• A new borrowal record is created successfully with 'Borrowed' status. • A success message is displayed. • The availability status of the book is updated to 'false' (unavailable).
TC_REG_FLOW_002	Member can view their own borrowal history	• Precondition: A member has at least one active or past borrowal record. • Login as the Member. • Navigate to the Borrowal Management page (/borrowals).	• The page displays a list of borrowal records. • The list contains ONLY the borrowals belonging to the logged-in member.
TC_REG_FLOW_003	Librarian can update a borrowal status (e.g., return a book)	• Precondition: A book has been borrowed by a member. • Login as Librarian. • Navigate to Borrowal Management. • Find the active borrowal record and edit it. • Change the status from 'Borrowed' to 'Returned'. • Save the changes.	• The borrowal record's status is successfully updated to 'Returned'. • The corresponding book's availability status is updated back to 'true' (available).

1.3.6 Installation and Deployment Testing

This testing ensures that the application can be installed and deployed successfully in the target environments (development, test, production) using the provided Docker configuration.

Table 1.22: Installation & Deployment Test Cases

Test ID	Description	Execution Steps	Expected Results	Status	Notes
TC-IDT-001	Verify Docker Environment Build and Launch	1. Navigate to the project root directory. 2. Run the command `docker-compose up -build -d`. 3. Check container status with `docker ps`.	All containers (lms-server, lms-client, lms-db) are running without errors. The client is accessible at `http://localhost:3000` and the server logs show a successful connection to the database.	Pass	
TC-IDT-002	Verify Database Initialization and Seeding	1. After a fresh launch, connect to the MongoDB container. 2. Access the `lms` database. 3. List collections and query the `users` collection.	The database contains all required collections as defined by the Mongoose models. The `users` collection is populated with the initial admin user from the seeding script.	Pass	
TC-IDT-003	Post-Deployment Smoke Test (Librarian)	1. Open a browser and navigate to `http://localhost:3000`. 2. Log in with Librarian credentials. 3. Navigate to the Dashboard, Books, and Users pages.	Login is successful. The user is redirected to the Dashboard. All navigated pages (Dashboard, Books, Users) load correctly without critical errors and display their respective UI components.	Pass	
TC-IDT-004	Post-Deployment Acceptance Test (Member)	1. In a new browser session, log in with Member credentials. 2. Navigate to the Books page and view a book's details. 3. Navigate to the Borrowals page to view personal history.	Login is successful. The user is redirected to the Books page. The user can view the list of books and their details. The borrowal history page loads correctly, displaying a message that no borrowals are found (for a new user).	Pass	
TC-IDT-005	Verify Docker Build Process	1. Run `docker-compose build -no-cache` from the root directory.	The Docker images for both the `server` and `client` services are built successfully without any errors during dependency installation or code copying.	Pass	

Continued on next page

Table 1.22 – continued from previous page

Test ID	Description	Execution Steps	Expected Results	Status	Notes
TC-IDT-006	Verify Graceful Shutdown	1. From the root directory, run the command `docker-compose down`. 2. Run `docker ps -a` to check for remaining containers.	All application containers are stopped and removed. The custom network created by Docker Compose is also removed. No orphaned resources remain.	Pass	

1.3.7 Static Testing

Static testing is performed without executing the application code. It includes reviews of documentation and source code.

- **Code Reviews:** Manual inspection of source code to identify potential defects, security vulnerabilities, and deviations from coding standards.
- **Walkthroughs:** The development team presents the code to the testing team to explain its logic and flow.
- **Documentation Review:** Review of the SRS, test plan, and other project documents for clarity, completeness, and consistency.

1.3.7.1 Documentation Review

All key project documents were subjected to a thorough review process. The review focused on identifying ambiguities, inconsistencies, and missing information. This process ensures that the entire team operates from a shared, unambiguous understanding of the project's goals and specifications.

- **Software Requirements Specification (SRS):** The SRS document was the cornerstone of our review process. It was meticulously examined to ensure that all functional and non-functional requirements were clearly defined, testable, and free from contradiction. Feedback from this review was used to refine the requirements, which in turn guided development and the creation of dynamic test cases.
- **Test Plan and Test Cases:** All documented test cases (including those for functional, API, integration, and non-functional testing) were reviewed to ensure they provided adequate coverage of the requirements. The review verified that test steps were clear, expected outcomes were precise, and traceability to the SRS was established.
- **Tasks and Responsibilities Matrix:** The `Tasks.tex` document was reviewed to ensure a clear allocation of duties for all testing activities, confirming that every aspect of the test plan had a designated owner.

1.3.7.2 Code Review and Walkthroughs

Source code was systematically reviewed to enforce quality and adherence to best practices.

- **Code Reviews:** Peer reviews were conducted for all major components of the application. Developers reviewed each other's code, focusing on logic, efficiency, error handling, security, and compliance with our established coding standards. This process was instrumental in identifying subtle bugs and improving code clarity.

- **Walkthroughs:** For complex workflows, such as the book borrowing and return process, developers conducted walkthroughs. In these sessions, the author of the code explained its structure and logic to other team members, who would ask questions and provide feedback. This method proved highly effective for verifying business logic and ensuring the implementation correctly reflected the use cases defined in the SRS.

1.3.7.3 Static Testing Test Cases and Findings

The following table summarizes the key activities performed during static testing and their outcomes. Unlike dynamic tests, these "test cases" represent structured review tasks.

Table 1.23: Static Testing Cases and Results

Test ID	Test Area	Objective	Outcome / Findings
TC-STATIC-001	SRS Document Review	Verify that all functional requirements are clear, complete, consistent, and testable.	✓ The requirements were found to be mostly clear. Minor ambiguities in the book return policy (Use Case UC-003) were identified and clarified with the team. Traceability links to test cases were established.
TC-STATIC-002	Test Plan and Test Cases Review	Ensure comprehensive test coverage for all requirements specified in the SRS. Verify clarity of test steps and expected results.	✓ Confirmed that all major use cases were covered by at least one functional or integration test. Some test case descriptions were enhanced for better clarity.
TC-STATIC-003	Tasks & Responsibilities Review	Confirm that all testing roles and responsibilities are clearly defined and assigned.	✓ The roles defined in `Tasks.tex` were confirmed to be clear and comprehensive, ensuring no gaps in test ownership.
TC-STATIC-004	Code Review: `authController.js`	Inspect authentication logic for potential security vulnerabilities (e.g., error message information disclosure) and adherence to best practices.	✓ The review identified that initial error messages for failed logins could potentially reveal whether a username exists. The logic was updated to return a generic "Invalid credentials" message in all failure cases.
TC-STATIC-005	Code Review: `borrowalControllerjs`	Verify the logic for borrowing, returning, and checking the status of books. Ensure correct state management and handling of edge cases (e.g., borrowing an unavailable book).	✓ The core logic was sound. A recommendation was made to add more detailed logging for borrowal transactions to facilitate easier debugging in the future.

Continued on next page

Table 1.23 – continued from previous page

Test ID	Test Area	Objective	Outcome / Findings
TC-STATIC-006	Code Review: Frontend Components (`/client/src/sections`)	Ensure React components follow best practices for state management, props handling, and separation of concerns. Check for UI consistency.	✓ Reviews led to minor refactoring in several components to improve reusability (e.g., creating a generic dialog component). Consistent error handling via `errorHandler.js` was verified.
TC-STATIC-007	Code Review: Database Models (`/server/models`)	Review Mongoose schemas for correctness, validation rules, and indexing to ensure data integrity and performance.	✓ Confirmed that appropriate validation (e.g., `required`, `enum`) was in place. A missing index on the `email` field of the User model was identified and added to improve query performance.
TC-STATIC-008	Walkthrough: User Registration & Login Workflow	Walk through the end-to-end code path from the frontend form submission to the backend API, controller logic, and database interaction.	✓ The walkthrough confirmed that all team members understood the flow of data and the role of JWT tokens in session management. It clarified the interaction between Passport.js and the custom controller logic.
TC-STATIC-009	Walkthrough: Book Management (CRUD)	Present the code for adding, updating, and deleting books, including referential integrity checks with authors and genres.	✓ The walkthrough helped identify a potential race condition if two librarians attempted to delete the same author simultaneously. While a low risk, it highlighted an area for future improvement with more robust transaction management.

1.4 Test Environment and Tools

All testing activities are conducted within a controlled and reproducible environment, managed by Docker. This ensures consistency between development, testing, and production setups.

Table 1.24: Testing Tools and Frameworks

Tool/Framework	Purpose
Docker	Containerization of the application, database (MongoDB), and testing environments. Used to ensure consistency across all stages.
Node.js	The runtime environment for the backend application and test execution.

Continued on next page

Table 1.24 – continued from previous page

Tool/Framework	Purpose
Jest	The primary framework for unit and integration testing of the backend. Provides test running, assertion, and mocking capabilities.
Supertest	An HTTP assertion library used with Jest for API integration testing.
Cypress	The framework for end-to-end (E2E) system testing, browser compatibility testing, and parts of regression testing.
k6	A modern load testing tool used for performance and stress testing of the API endpoints.
MongoDB	The NoSQL database used by the application. A separate test database instance is used for automated testing.
Git / GitHub	Version control system for source code and test scripts. Used for collaboration and tracking changes.

1.5 Evaluation and Reporting

The evaluation of the system's quality is a continuous process involving test execution, defect management, and reporting.

- **Test Execution:** Tests are executed automatically via npm scripts (e.g., `npm test`) and on a continuous integration (CI) server (conceptual).
- **Defect Management:** A systematic approach is used to manage defects. When a test fails, a defect is logged with details including steps to reproduce, severity, and priority. The defect is then assigned, fixed, and re-tested.
- **Test Reporting:** After each test cycle, reports are generated. These include Jest test results (see Figure 1.5) and code coverage reports (Figure 1.2). These reports provide a clear overview of the application's health to the entire team.

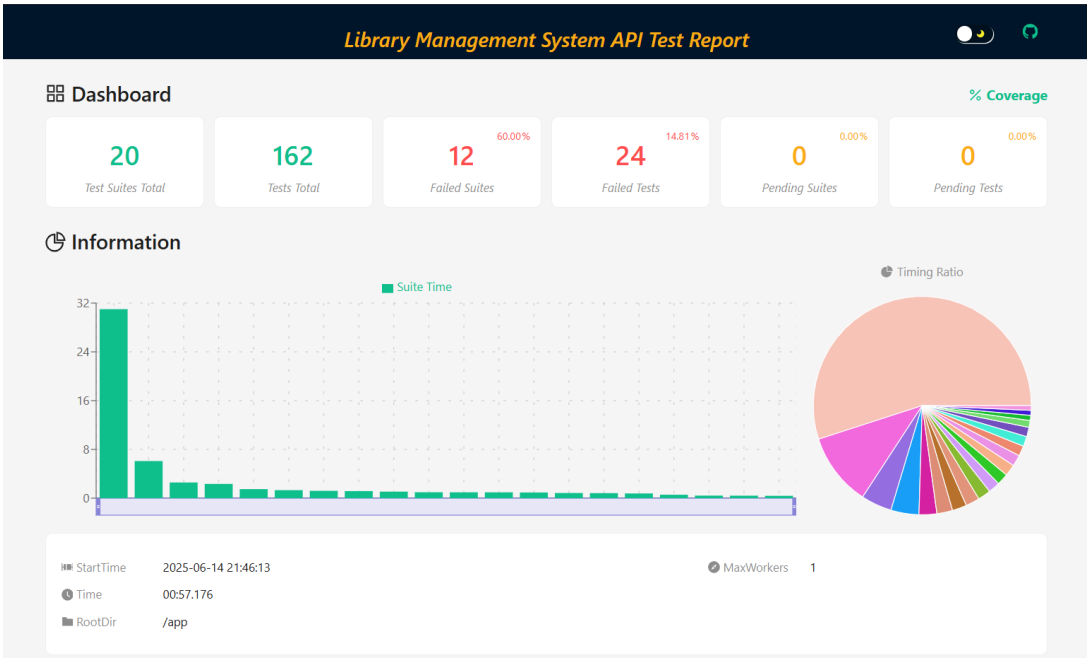

Figure 1.5: Jest Test Execution Report